Statistical Machine Learning (BE4M33SSU)
EM algorithm; Bayesian learning

Czech Technical University in Prague

- Unsupervised generative learning
- Expectation Maximisation algorithm
- Bayesian inference

Unsupervised generative learning

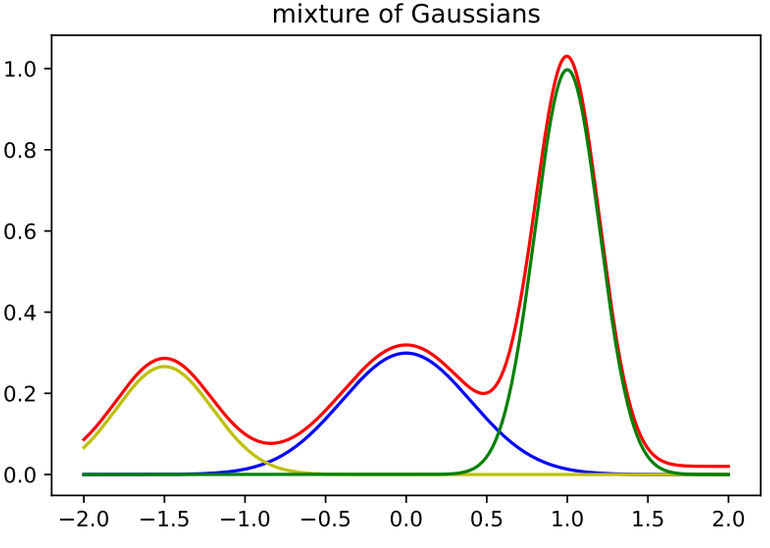
- The joint p.d. $p_\theta(x, y)$, $\theta \in \Theta$ is known up to the parameter $\theta \in \Theta$.
- We are given training data $\mathcal{T}^m = \{x^j \in \mathcal{X} \mid j = 1, \ldots, m\}$ i.i.d. generated from p_{θ^*}.

Can we estimate the parameter θ without ever seeing the hidden states y?

Example 1 (Mixture of Gaussians).
We observe data $x \in \mathbb{R}$ generated from a mixture of $|\mathcal{Y}|$ Gaussians

$$p(x) = \sum_{y \in \mathcal{Y}} p(y) p(x \mid y) = \sum_{y \in \mathcal{Y}} p(y) \frac{1}{\sqrt{2\pi}\sigma_y} e^{-\frac{(x-\mu_y)^2}{2\sigma_y^2}}$$

Can we estimate the parameters $p(y)$, μ_y, σ_y from given training data $\mathcal{T}^m = \{x^j \in \mathcal{X} \mid j = 1, \ldots, m\}$?



Unsupervised generative learning

Example 2 (Generating handwritten digits)**.**
Our training set consists of images of handwritten digits (MNIST). We want to design and train a model for generating such images. We consider a model

$$p(x,z) = p_\theta(x\,|\,z)p(z),$$

where $x \in \mathbb{R}^{h\times w}$ is an image and $z \in \mathbb{R}^n$ is a vector of latent variables encoding shapes and writing styles. We fix a simple prior distribution $p(z)$ on the latent space, e.g. $\mathcal{N}(0,\mathbb{I})$, and a parametric model $p_\theta(x\,|\,z)$, e.g. $\mathcal{N}\big(\mu(z,\theta),\sigma^2\mathbb{I}\big)$, where $\mu(z,\theta)$ is a parametrised mapping $z \in \mathbb{R}^n \mapsto x \in \mathbb{R}^{h\times w}$.
Can we estimate the parameter θ without ever seeing the latent states z?

Unsupervised generative learning

Given a parametric family of distributions $\{p_\theta(x,y) \mid \theta \in \Theta\}$ and a training set $\mathcal{T}^m = \left\{x^j \in \mathcal{X} \,\middle|\, i = 1,\ldots,m\right\}$, we want to estimate the model parameter θ by the maximum likelihood estimator (MLE)

$$e_{ML}(\mathcal{T}^m) = \arg\max_{\theta\in\Theta} L(\theta) = \arg\max_{\theta\in\Theta} \mathbb{E}_{x\sim\mathcal{T}^m}\Big[\log p_\theta(x)\Big] = \arg\max_{\theta\in\Theta} \mathbb{E}_{x\sim\mathcal{T}^m}\Big[\log \sum_{y\in\mathcal{Y}} p_\theta(x,y)\Big]$$

- If θ is a single parameter or a vector of homogeneous parameters $\Rightarrow$ maximise the log-likelihood directly by gradient ascent (provided it is differentiable in θ).
- If θ is a collection of heterogeneous parameters $\Rightarrow$ apply the **Expectation Maximisation Algorithm** (Schlesinger, 1968, Sundberg, 1974, Dempster, Laird, and Rubin, 1977)
- The EM algorithm transforms the original unsupervised problem into a series of supervised MLE tasks, which are typically much easier to solve.

The Expectation Maximisation Algorithm

Start with a suitably chosen $\theta^{(0)}$ and iterate the following steps until convergence

E-step Fix the current $\theta^{(t)}$ and compute

$$\alpha_x^{(t)}(y) = p_{\theta^{(t)}}(y \mid x).$$

M-step Fix the current $\alpha^{(t)}$, use them as "soft" labels and solve the MLE task.

$$\theta^{(t+1)} = \arg\max_{\theta \in \Theta} \mathbb{E}_{\mathcal{T}^m}\left[\sum_{y \in \mathcal{Y}} \alpha_x^{(t)}(y) \log p_\theta(x, y)\right]$$

This is equivalent to solving the MLE for annotated training data.

Claims:

- The sequence of likelihood values $L(\theta^{(t)}) = \mathbb{E}_{x \sim \mathcal{T}^m}[\log p_{\theta^{(t)}}(x)]$, $t = 1, 2, \ldots$ is increasing.
- The sequence of $\alpha_x^{(t)}$, $t = 1, 2, \ldots$ is convergent.

There is **no guarantee** that the EM algorithm converges to a global maximum of the log-likelihood. This underlines the importance of a suitable initialisation.

The Expectation Maximisation Algorithm

Example 3 (Fitting mixture of Gaussians)**.** Given training data $\mathcal{T}^m = \{x^j \in \mathbb{R} \mid j = 1, \ldots, m\}$, estimate the parameters $p(y)$, μ_y, σ_y of the GMM:

$$p_\theta(x) = \sum_{y \in \mathcal{Y}} p(y) p(x \mid y) = \sum_{y \in \mathcal{Y}} p(y) \frac{1}{\sqrt{2\pi}\sigma_y} e^{-\frac{(x-\mu_y)^2}{2\sigma_y^2}}$$

Start with a suitably chosen $\theta^{(0)} = (p^{(0)}(y), \mu_y^{(0)}, \sigma_y^{(0)})$ and iterate until convergence:

E-step Fix the current $\theta^{(t)}$ and compute

$$\alpha_x^{(t)}(y) = p_{\theta^{(t)}}(y \mid x) = \frac{p^{(t)}(y)}{\sqrt{2\pi}\sigma_y} e^{-(x-\mu_y^{(t)})^2/(2\sigma_y^{(t)})^2} \Big/ \left(\sum_{\bar{y} \in \mathcal{Y}} \frac{p^{(t)}(\bar{y})}{\sqrt{2\pi}\sigma_{\bar{y}}^{(t)}} e^{-(x-\mu_{\bar{y}}^{(t)})^2/(2\sigma_{\bar{y}}^{(t)})^2} \right)$$

M-step Fix the current $\alpha^{(t)}$, use them as "soft" labels and solve the MLE task.

$$(p^{(t+1)}(y), \mu_y^{(t+1)}, \sigma_y^{(t+1)}) = \operatorname*{arg\,max}_{p(y), \mu_y, \sigma_y} \mathbb{E}_{\mathcal{T}^m} \left[\sum_{y \in \mathcal{Y}} \alpha_x^{(t)}(y) \log p_\theta(x, y) \right]$$

which has a closed form solution:

$$p^{(t+1)}(y) = \frac{\mathbb{E}_{\mathcal{T}^m}[\alpha_x^{(t)}(y)]}{\sum_{\bar{y} \in \mathcal{Y}} \mathbb{E}_{\mathcal{T}^m}[\alpha_x^{(t)}(\bar{y})]}, \quad \mu_y^{(t+1)} = \frac{\mathbb{E}_{\mathcal{T}^m}[\alpha_x^{(t)}(y) x]}{\mathbb{E}_{\mathcal{T}^m}[\alpha_x^{(t)}(y)]}, \quad \left(\sigma_y^{(t+1)}\right)^2 = \frac{\mathbb{E}_{\mathcal{T}^m}[\alpha_x^{(t)}(y)\left(x - \mu_y^{(t+1)}\right)^2]}{\mathbb{E}_{\mathcal{T}^m}[\alpha_x^{(t)}(y)]}$$

The Expectation Maximisation Algorithm

- Introduce auxiliary variables $\alpha_x(y) \geqslant 0$, for each $x \in \mathcal{T}^m$, s.t. $\sum_{y \in \mathcal{Y}} \alpha_x(y) = 1$
- Construct a lower bound of the log-likelihood $L(\theta) \geqslant L_B(\theta, \alpha)$
- Maximise this lower bound by block-wise coordinate ascent.

Construct the bound using the Jensen's inequality:

$$L(\theta) = \mathbb{E}_{\mathcal{T}^m}\Big[\log \sum_{y \in \mathcal{Y}} p_\theta(x, y)\Big] = \mathbb{E}_{\mathcal{T}^m}\Big[\log \sum_{y \in \mathcal{Y}} \frac{\alpha_x(y)}{\alpha_x(y)} p_\theta(x, y)\Big] \geqslant$$

$$L_B(\theta, \alpha) = \mathbb{E}_{\mathcal{T}^m} \sum_{y \in \mathcal{Y}} \Big[\alpha_x(y) \log p_\theta(x, y) - \alpha_x(y) \log \alpha_x(y)\Big]$$

The following equivalent representation shows the difference between $L(\theta)$ and $L_B(\theta, \alpha)$:

$$L_B(\theta, \alpha) = \mathbb{E}_{\mathcal{T}^m}\big[\log p_\theta(x)\big] - \mathbb{E}_{\mathcal{T}^m}\big[D_{KL}(\alpha_x(y) \,\|\, p_\theta(y \,|\, x))\big]$$

We see that the lower bound is tight if $\alpha_x(y) = p_\theta(y \,|\, x)$ holds $\forall x$ and $\forall y$.

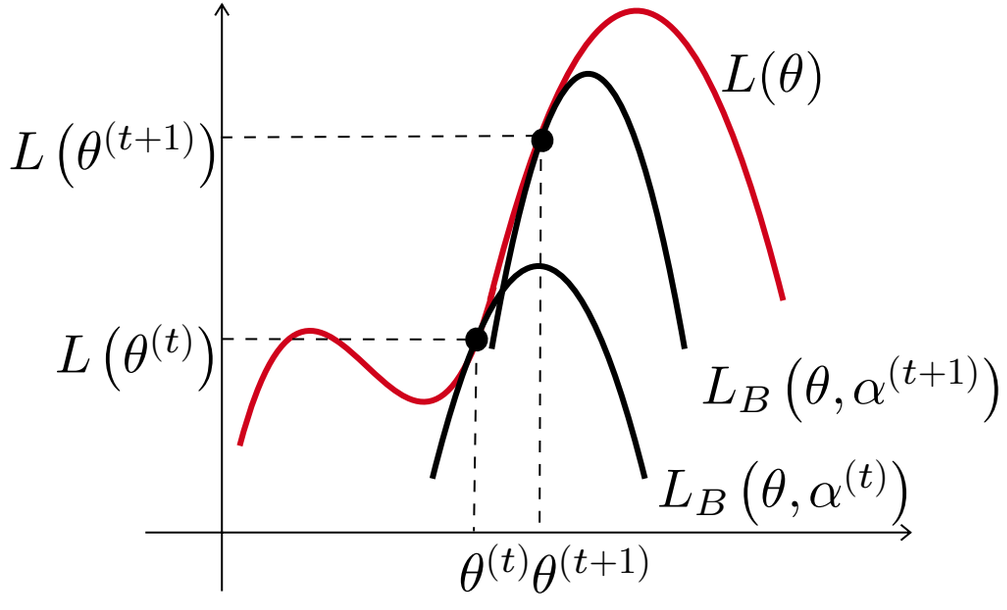
The Expectation Maximisation Algorithm

- The EM implements the block-wise coordinate ascent of the lower bound

$$L(\theta) \geqslant L_B(\theta, \alpha) = \mathbb{E}_{\mathcal{T}^m} \sum_{y \in \mathcal{Y}} \Big[\alpha_x(y) \log p_\theta(x, y) - \alpha_x(y) \log \alpha_x(y) \Big], \qquad \forall \theta, \alpha$$

E-step Fix the current $\theta^{(t)}$ and compute

$$\alpha_x^{(t)}(y) = \underset{\substack{\alpha_x(y) \geq 0 \\ \sum_y \alpha_x(y) = 1}}{\arg\max} L_B(\theta^{(t)}, \alpha) = p_{\theta^{(t)}}(y \mid x)$$



M-step Fix the current $\alpha^{(t)}$ and compute

$$\begin{aligned} \theta^{(t+1)} &= \underset{\theta \in \Theta}{\arg\max}\, L_B(\theta, \alpha^{(t)}) \\ &= \underset{\theta \in \Theta}{\arg\max}\, \mathbb{E}_{\mathcal{T}^m} \Big[\sum_{y \in \mathcal{Y}} \alpha_x^{(t)}(y) \log p_\theta(x, y) \Big] \end{aligned}$$

The Expectation Maximisation Algorithm

Additional reading:

Schlesinger, Hlavac, Ten Lectures on Statistical and Structural Pattern Recognition, Chapter 6, Kluwer 2002 (also available in Czech)

Thomas P. Minka, Expectation-Maximization as lower bound maximization, 1998 (short tutorial, available on the internet)

Bayesian inference

Motivation:

- Both, ERM and generative learning by MLE are consistent under the respective regularity assumptions. Their estimation errors $R(h_m) - R(h_{\mathcal{H}})$ and $\|\theta_m - \theta^*\|$ are small in the limit of large training data sizes m.
- On the other hand, their estimates h_m and θ_m can deviate substantially from the respective optimal predictor/model in case of small training data sizes.
- Models should be based on our knowledge about the problem. We do not want to restrict the complexity of the model $p_\theta(x, y)$, $\theta \in \Theta$ just because we have only a small amount of training data.
- Deciding for a single model $\theta_m = e_{ML}(\mathcal{T}^m)$ might be sub-optimal in such situations.

Idea:

- Incorporate an additional prior, specifically a prior on the distribution of the parameters $\theta \in \Theta$.
- Rather than selecting a single model, take all models into account simultaneously when designing the predictor, i.e. use a weighted mixture of the models.

$$p(x, y) = \sum_{k=1}^{K} \alpha_k(\mathcal{T}^m)\, p_{\theta_k}(x, y)$$

Bayesian inference

Bayesian inference:

Interpret the unknown parameter $\theta \in \Theta$ as a **random** variable.

- Data distribution: parametric family of models $\{p(x, y \,|\, \theta) \,|\, \theta \in \Theta\}$,
- Prior distribution $p(\theta)$ on Θ.

The prior distribution $p(\theta)$ and i.i.d. training data $\mathcal{T}^m = \big\{(x_j, y_j) \,\big|\, j = 1, \ldots, m\big\}$ define a *posterior parameter distribution* $p(\theta \,|\, \mathcal{T}^m)$, given by

$$p(\theta \,|\, \mathcal{T}^m) = \frac{p(\theta) p(\mathcal{T}^m \,|\, \theta)}{p(\mathcal{T}^m)} \quad \text{with} \quad p(\mathcal{T}^m \,|\, \theta) = \prod_{i=1}^{m} p(x^i, y^i \,|\, \theta).$$

The probability $p(\mathcal{T}^m)$ is obtained by integrating over θ, i.e. $p(\mathcal{T}^m) = \int p(\theta) p(\mathcal{T}^m \,|\, \theta)\, d\theta$ and does not depend on θ.

Notice that $p(\mathcal{T}^m \,|\, \theta)$ represents the likelihood of the data $\mathcal{T}^m$ given the parameter θ.

Notice that the posterior distribution $p(\theta \,|\, \mathcal{T}^m) \propto p(\mathcal{T}^m \,|\, \theta)\, p(\theta)$ interpolates between the situation without any training data, i.e. $m = 0$ and the likelihood of training data for $m \to \infty$ when $p(\theta \,|\, \mathcal{T}^m) \approx \text{constant} \cdot p(\mathcal{T}^m \,|\, \theta)$.

Bayesian inference: MAP decision

Let us use $p(\theta\,|\,\mathcal{T}^m)$, but decide for a single value of θ by using the MAP criterion,

$$\theta_m = \arg\max_{\theta\in\Theta} p(\theta\,|\,\mathcal{T}^m) = \arg\max_{\theta\in\Theta} p(\mathcal{T}^m\,|\,\theta)\,p(\theta) = \arg\max_{\theta\in\Theta}\left[\sum_{(x,y)\in\mathcal{T}^m} \log p(x,y\,|\,\theta) + \log p(\theta)\right]$$

This results in an ML estimate with an additional regulariser

$$\theta_m = \arg\max_{\theta\in\Theta}\left[\frac{1}{m}\sum_{(x,y)\in\mathcal{T}^m} \log p(x,y\,|\,\theta) + \frac{1}{m}\log p(\theta)\right]$$

Example 4. We want to learn a DNN classifier with squashing activation functions (e.g. tanh or sigmoid). Assuming a Gaussian prior for the network weights, i.e. $w \sim \mathcal{N}(0, \sigma\, I)$, we get the learning objective

$$\frac{1}{m}\sum_{(x,y)\in\mathcal{T}^m} \log p(y\,|\,x;\,w) - \frac{1}{2m\sigma^2}\|w\|^2 \to \max_w$$

This enforces a considerable fraction of neurons to have small weights and thus also small activations. They will therefore operate in a semi-linear regime.

Bayesian inference

The Bayesian approach uses the posterior distribution $p(\theta \,|\, \mathcal{T}^m) \propto p(\mathcal{T}^m \,|\, \theta)\, p(\theta)$ to construct model mixtures and predictors. Consider the posterior probability to observe a pair (x, y) by marginalising over $\theta \in \Theta$:

$$p(x, y \,|\, \mathcal{T}^m) = \frac{1}{p(\mathcal{T}^m)} \int_\Theta p(\mathcal{T}^m \,|\, \theta)\, p(\theta)\, p(x, y \,|\, \theta)\, d\theta$$

This is a **mixture of distributions** with mixture weights $\alpha_m(\theta) \propto p(\mathcal{T}^m \,|\, \theta)\, p(\theta)$.

The Bayes optimal predictor w.r.t. 0/1 loss for this model mixture is

$$h(x, \mathcal{T}^m) = \operatorname*{arg\,max}_{y \in \mathcal{Y}} \int_\Theta \underbrace{p(\theta)\, p(\mathcal{T}^m \,|\, \theta)}_{\alpha_m(\theta)}\, p(x, y \,|\, \theta)\, d\theta = \operatorname*{arg\,max}_{y \in \mathcal{Y}} \int_\Theta \alpha_m(\theta)\, p(x, y \,|\, \theta)\, d\theta$$

Notice:

- the mixture weights $\alpha_m(\theta)$ interpolate between the situation without any training data, i.e. $m = 0$ and the likelihood of training data for $m \to \infty$.
- similar approaches for ERM lead to *Ensembling* methods.